Advanced Web Development

4 - Web Security with Open Authorisation (auth0 API) & Prisma ORM

Jeannette Chin

Recall last weeks

- REACT Basics – component, JSX, rendering
- Event Handling and State Management in REACT - useState hook
- Next.js Basics
  - Routing and Navigation
  - Client and Server Components
  - Data Fetching – Fetch and Axios API
- useEffect hook
- RTK for global state management

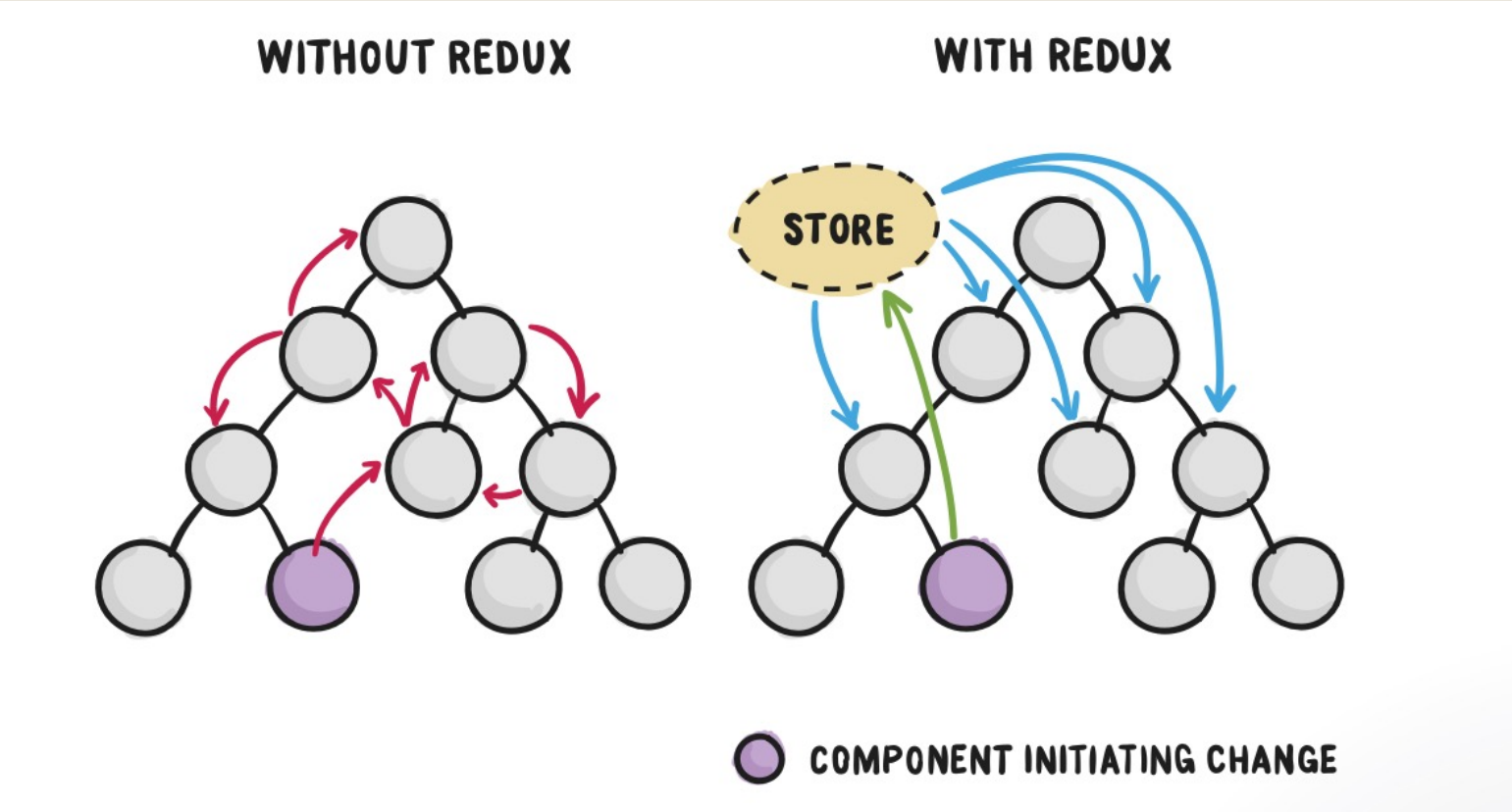
Learning Objectives

- Explain different types of Web Authentication methods
- Implement login and register UI
- Implement Google Provider using auth.js API
- Custom hook
- Prisma ORM to connect to PostgreSQL
- Explain API End points and security risks and defence solutions

Recall RTK



- Light weight State management tool written in JavaScript
- Standalone JS library
- Supports REACT and Next.js
- It allows you to manage your application’s state in a single place and keep changes in your application more predictable and traceable, making it easier to understand the changes happening in your application.
- But all of these benefits come with a set of challenges.
- Developers often argue that it is overly complicated what are otherwise simple tasks.



Source: https://jscrambler.com/images/redux-article-3-03.svg

Login and User Registration

- Very common tasks in Web development
- Security and data privacy
- Web Security – AAA (Authentication, Authorisation, Accountability)
  - Authentication
    - username & password, digital certificate
  - Authorisation
    - After successful authentication, then is the user authorised to do the following
      - Executing commands?
      - Access certain resources – e.g. certain directory in the filesystems
      - Information – e.g. user data?
  - Accountability
    - About records of what is going on within the whole application, who is accessing – when and what and for how long?

Authentication Types



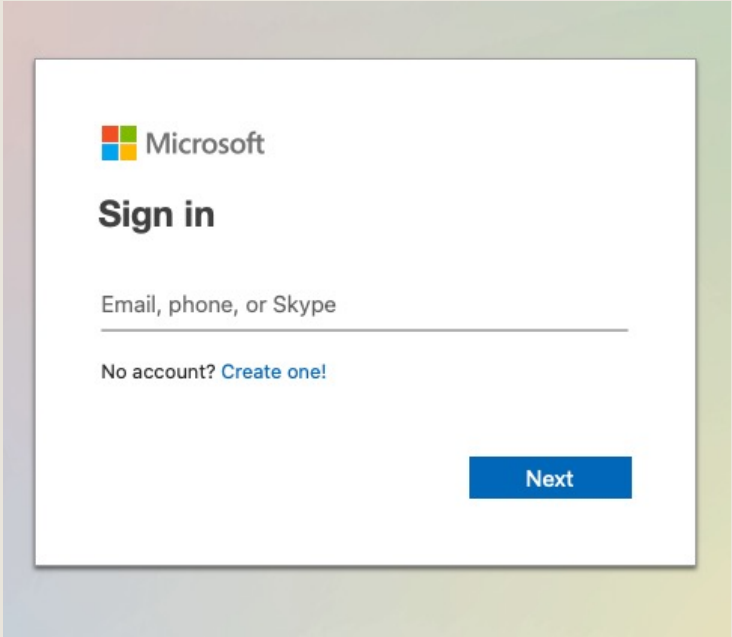
With a password



Without a password

Password-based Authentication

- The most familiar and widely used method for web applications
  - Users register with a unique identifier (typically an email or username) and a password.
  - The server compares the submitted password to a securely stored hash when logging in.
  - If the credentials match, the user is authenticated and granted access.



Password-based Authentication

- Advantages:
  - Quick to implement with mature libraries
  - Low infrastructure overhead
  - Easy to debug and support
  - Broad compatibility with existing systems
- Disadvantages:
  - Requires secure password hashing (e.g., bcrypt or Argon2) to avoid vulnerabilities
  - Needs layered defences (rate limiting, breach detection)
  - Generates high support volume from password resets and lockouts
  - Balancing strong policies with user experience can be difficult

Multi-Factor Authentication (MFA or 2FA)

- Strengthens app security
  - by requiring users to pass multiple verification steps, such as:
    - A code sent via SMS or email
    - A code generated by an authenticator app
    - A biometric check – fingerprint or facial recognition
  - Access is granted only after both authentication factors are verified.
- It has become a standard for applications involving sensitive data or privileged access.

Multi-Factor Authentication (MFA or 2FA)

- Advantages:
  - adds strong protection with minimal code overhead
  - Widely supported by third-party services and libraries
  - Shifts part of the security burden to external, verifiable factors
- Disadvantages:
  - Increases setup and testing complexity
  - Requires fallback strategies for lost devices or inaccessible second factors
  - May introduce friction during user onboarding
  - Needs secure backup or recovery flows to maintain usability

Token-based Authentication (digital, web)

- Go-to method for modern web apps and APIs.
- Instead of maintaining server-side sessions, it uses tokens as temporary digital credentials to verify user identity across requests.
- Stateless - use tokens as a secure means to authenticate API requests
- E.g. JSON Web Token (JWT)

JSON Web Token (JWT)

- A server creates a token that proves the client's identity and sends it to the client.
- JWT uses digital signatures to prove the token is legitimate.

JWT has three components:

1. Header: The header provides information about the JWT — what kind of token and which method /algorithm was used to digitally sign it
2. Payload: JSON data. JWT payloads for authentication include the user's identity , also information about the user's, server's, or API endpoint's permissions etc
3. Digital signature: Verify that the message has not changed in transit with a secure signature. Think of the digital signature as a tamper-proof seal on a new bottle of perfume.

Token-based Authentication (digital, web)

- Advantages:
  - Scale well
  - Simplifies session management in distributed systems
  - Reduces database load by removing the need for session lookups
  - Allows custom claims (roles, permissions, expiration) within tokens
  - Works seamlessly with JavaScript frameworks like React, Next.js, and Vue
- Disadvantages:
  - Requires secure token storage on the frontend to prevent XSS attacks
  - Token validation introduces a slight processing overhead per request
  - Managing expiration and refresh flows adds complexity
  - Revocation mechanisms (e.g., blacklists or short lifespans) require additional infrastructure

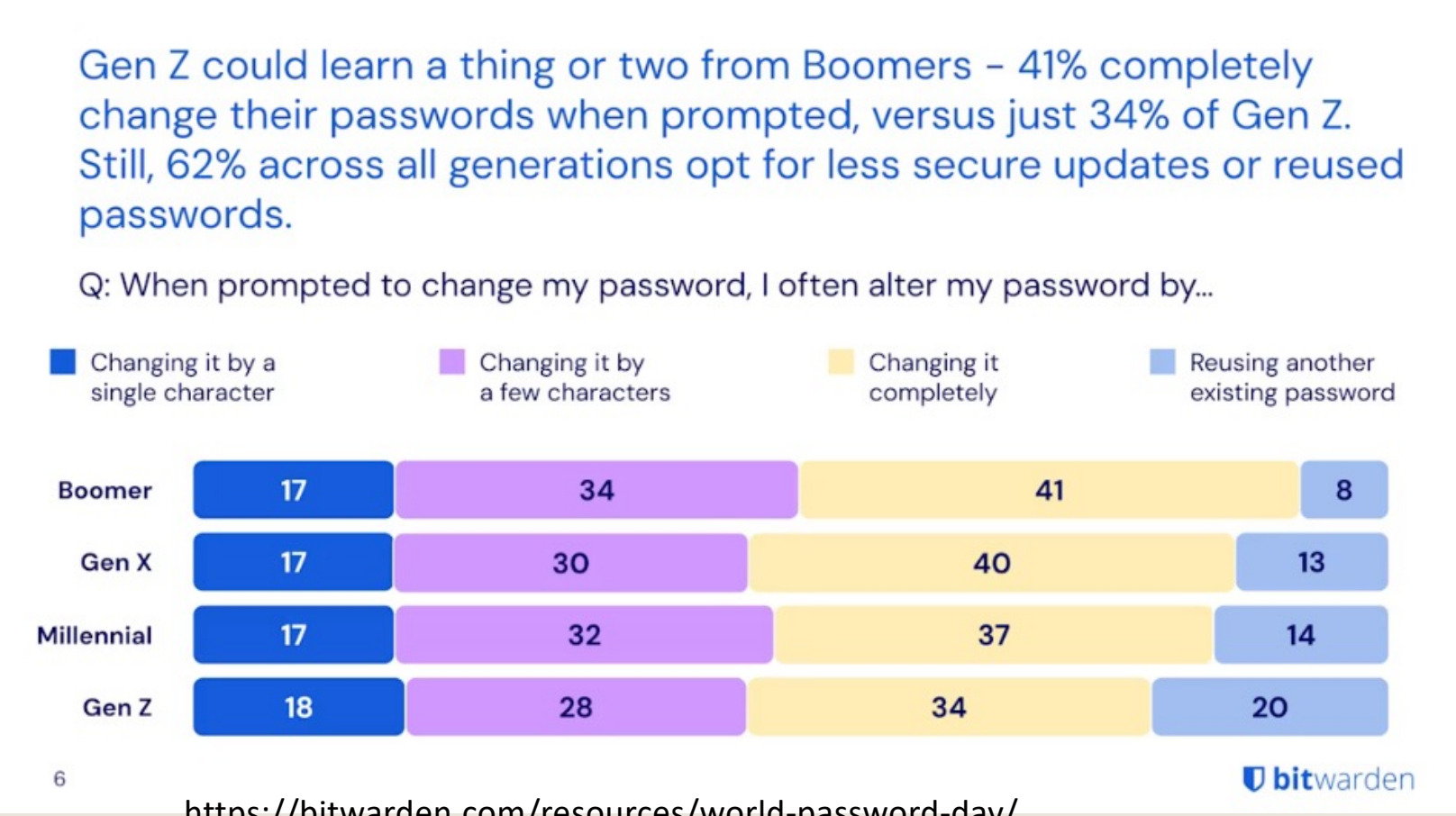
Other Authentication Methods

- Open Authorisation (OAuth) (Authorisation)
  - Utilise Open Standard Protocol for Secured Access Delegation
  - It is Authorisation Protocol (not Authentication) which allows users to share their private resources to a third party , Like login to a Web Application using Github/Twitter/ Gmail …etc
  - The concept of Access Tokens instead of credentials
- Single Sign On (SSO)
  - based on protocols such as OpenID Connect (OIDC)
  - allows the user to login with single set of credentials (username and password) from an existing account in one trusted service to sign-in to another new service.
  - Well-known service provider : Apple, Google, Facebook, X, Yahoo

Other Authentication Methods

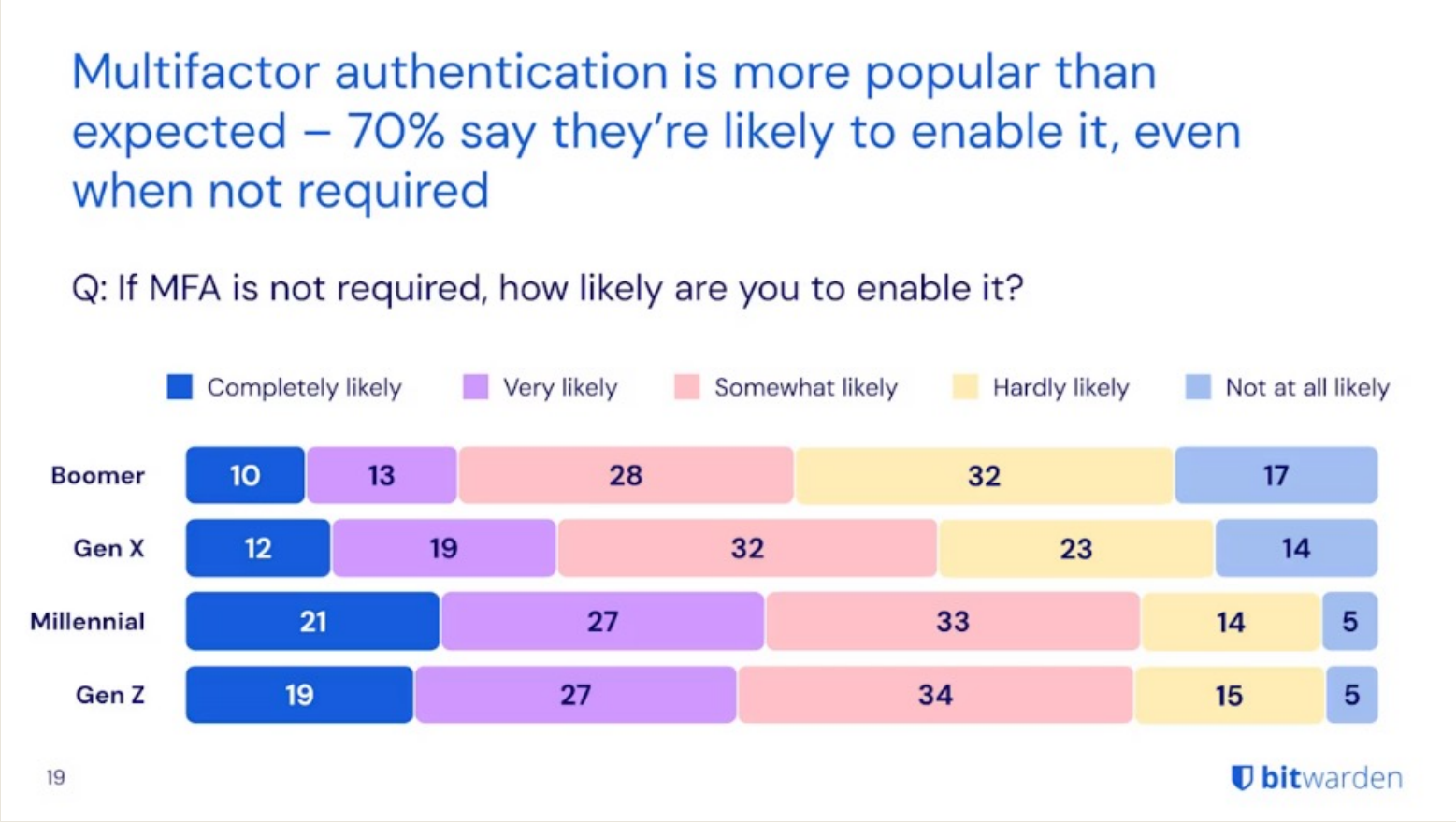
- Biometric Authentication
  - uses unique physical characteristics, such as fingerprints or facial features, as identity credentials.
  - These traits are difficult to replicate, making biometric authentication a powerful security layer for sensitive systems.
- Magic links
  - allows a user to log in by clicking a link that has been emailed to their registered address, rather than typing in their username and password.
- One-time passwords (OTPs)
  - the user doesn't need to create or remember a password
    - receive an SMS or email containing a single-use password to sign-in with
    - are asked to generate an OTP using an app

Password Fatigue



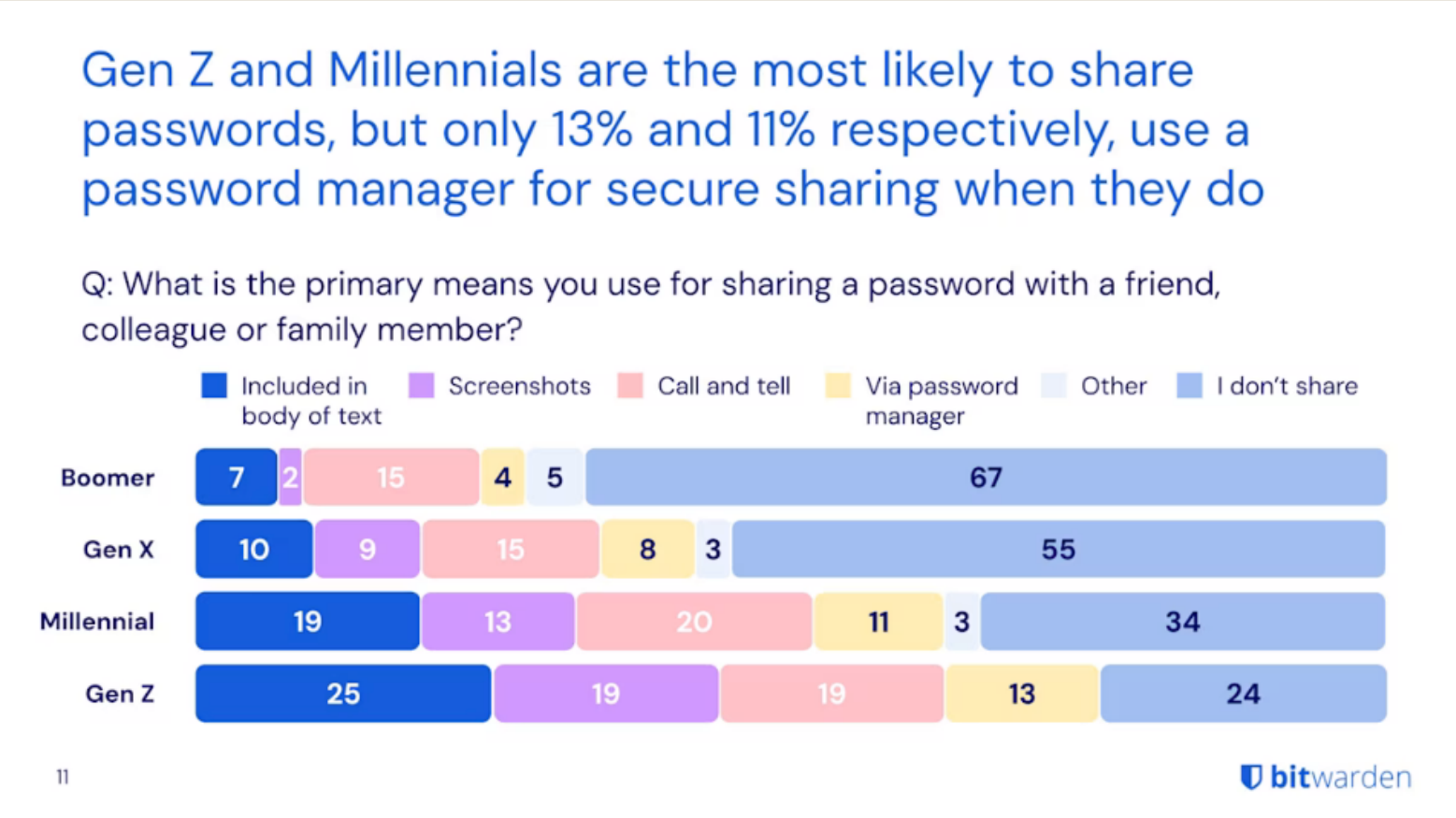
https://bitwarden.com/resources/world-password-day/

MFA gains Popularity



https://bitwarden.com/resources/world-password-day/

Sharing your password?



https://bitwarden.com/resources/world-password-day/

Authentication vs. Authorisation

- **Authentication — "Who are you?"** Verifying identity using credentials, magic links, or OAuth providers.
  - Output: An authenticated session token or JWT.
- **Authorisation — "What are you allowed to do?"** Verifying access permissions (e.g., standard user vs. admin role).
  - Executed via role-based access control.

Modern Threat Vectors & Defensive Solutions

- Threat Vector: Passkey / WebAuthn Shifts, Token Exfiltration , Cross-Site Request Forgery, Data Breach / DB Leaks
- Mechanism: Password-based phishing & leaks , XSS steals local Storage tokens , unauthorised commands via user browser, storing plain or weakly hashed passwords
- Defensive Solutions: Adopt biometric / hardware FIDO2 keys, store sessions in httpOnly Cookies, CSRF Tokens & SameSite Cookie flags, Bcrypt / Argon2id hashing or OAuth

Why HTTP-Only, Secure Cookies Matter

- Never store session JWTs in `localStorage` or `sessionStorage`
- JavaScript running on the page (e.g., compromised third-party scripts) can read them!
- **HTTP-Only cookies** are inaccessible to JavaScript, insulating session tokens from Cross-Site Scripting (XSS) attacks.

Auth.js (v5)

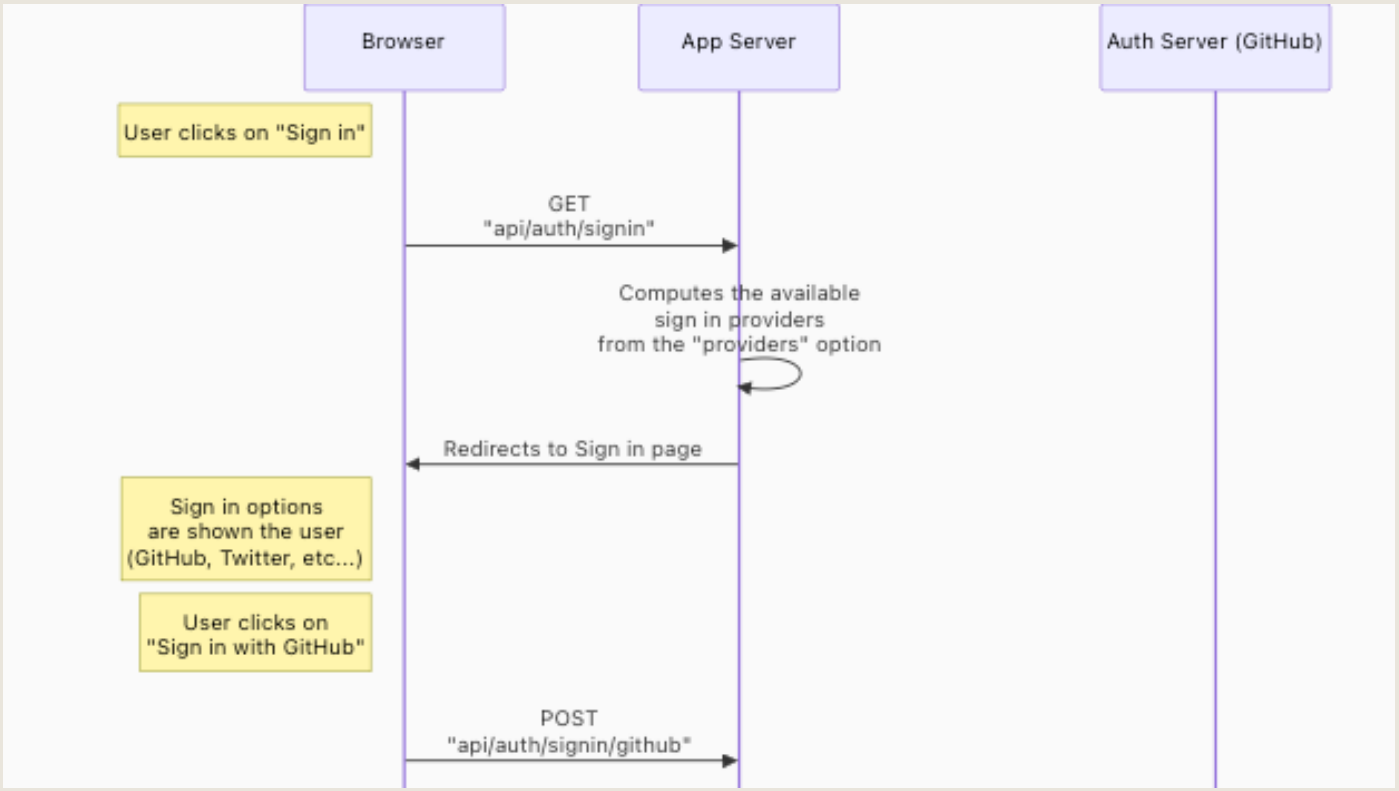
https://authjs.dev/getting-started and select Next.js

Auth.js

- Open-source
- Built-in support for OAuth, Email magic links, WebAuthn / passkeys and custom credentials https://authjs.dev/getting-started/authentication
- Support backend databases – MySQL, PostgreSQL, MongoDB etc
- Support typescript
- Support both JWT (default) and DB session strategies
  - HttpOnly cookie, encrypted with a secret key

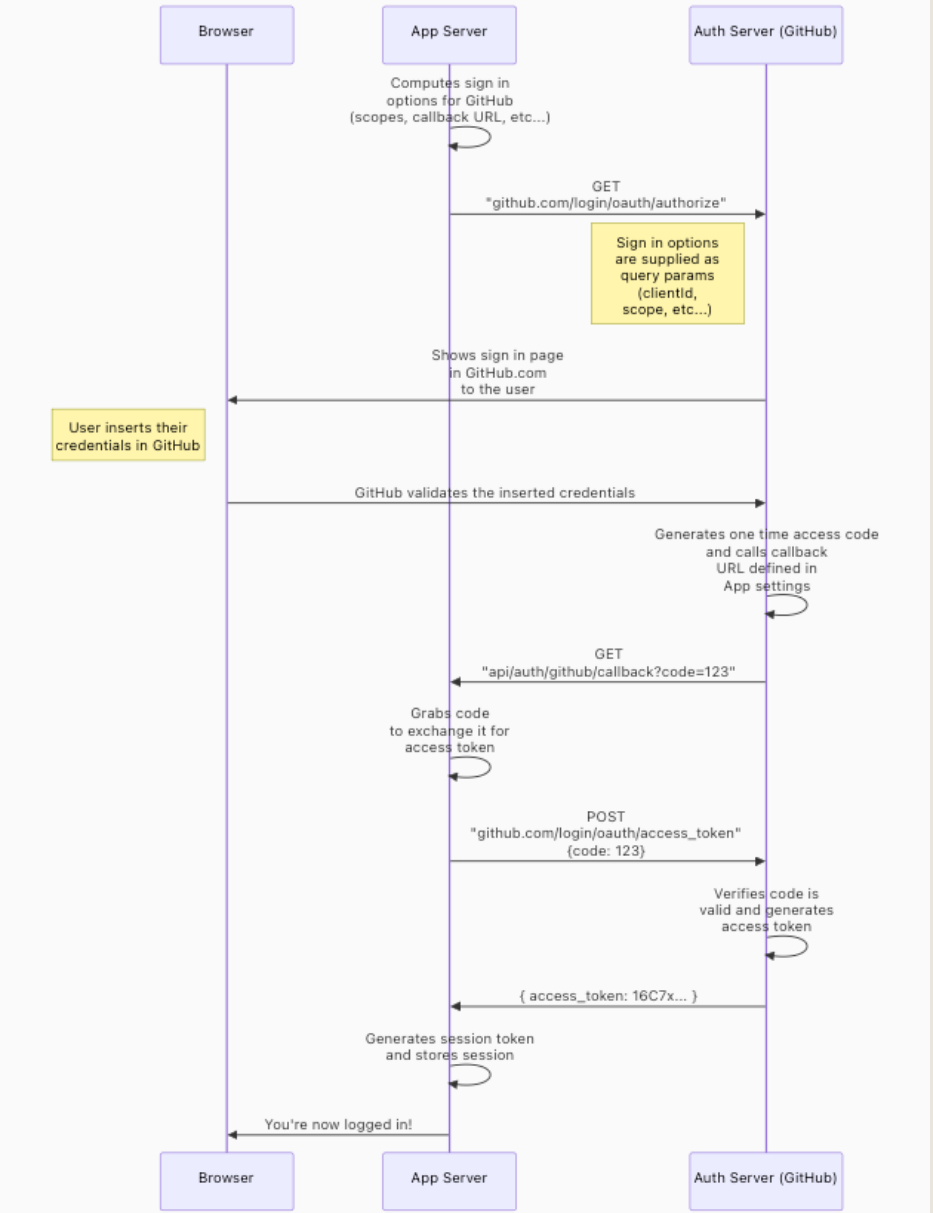
OAuth

- Using Github Provider



OAuth

- Using Github Provider



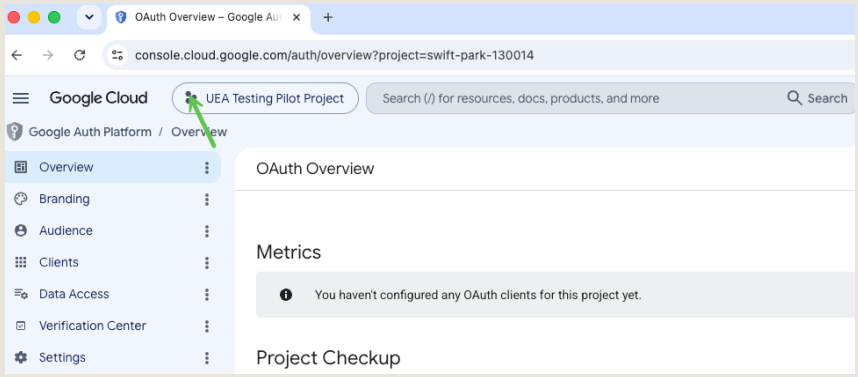
Teaching code – Part1

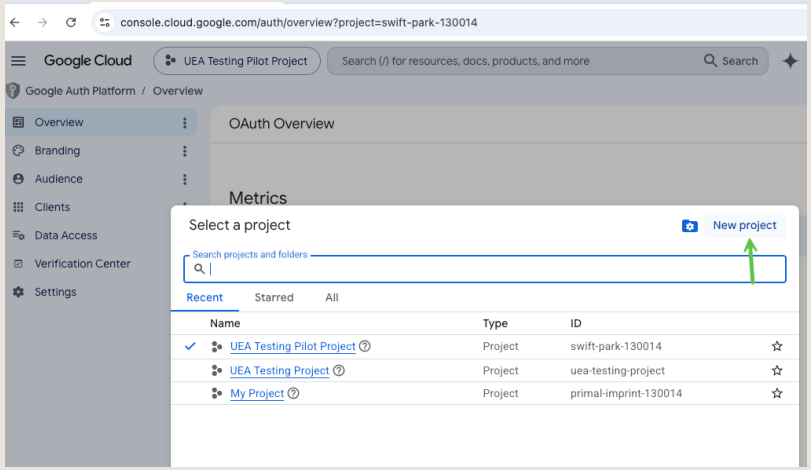
https://stugit.cmp.uea.ac.uk/jc/Adv-WebDev-W4-part1.git

Google Provider

Step 1: Create a Project in Google Cloud Console

- Navigate to the Google Cloud Console.
- Sign in with your Google account.
- In the top navigation bar, click the project selector dropdown (next to "Google Cloud") and click **New Project**.
- Name your project (e.g., NextAuth Demo) and click **Create**.

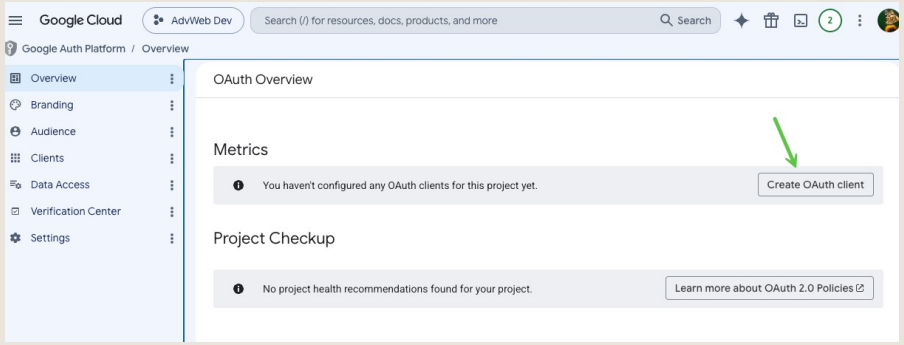


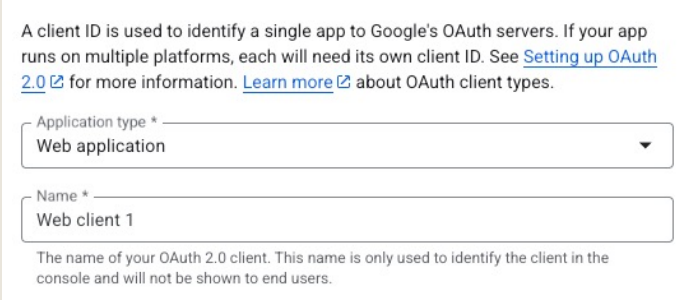


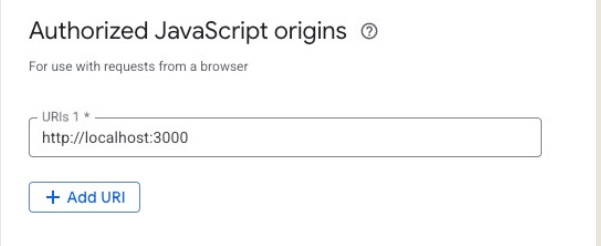
Google Provider

Step 2: Configure the OAuth

- Before Google gives you keys, it needs to know what your users will see when they sign in.
- Inside the project click Create **OAuth client** .
- Select Web Application
- Configure development URL







Google Provider

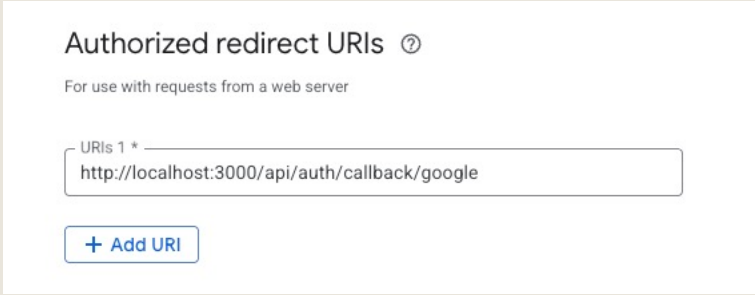
Step 2: Configure the OAuth

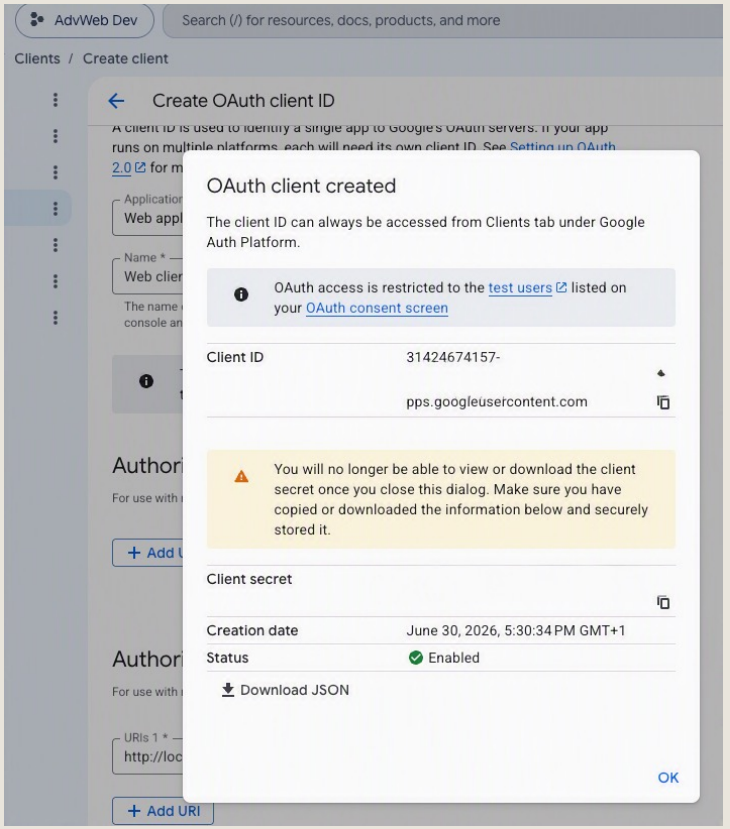
- Configure callback URL and click create

Critical: Auth.js expects this exact path (`/api/auth/callback/google`). A single missing slash or typo here will cause a `redirect_uri_mismatch` error!

- Copy the ClientID and client secret for use later

It is important to copy the Client ID and secret now as it will disappear after you click OK.





Google Provider

Step 3: Add Credentials to .env

- Add the keys to your project's environment file:

```
# Secret for signing cookies (Generate via: openssl rand -base64 32)
AUTH_SECRET="your-generated-auth-secret"

# Google Credentials from Google Cloud Console
AUTH_GOOGLE_ID="1234567890-abcdefg.apps.googleusercontent.com"
AUTH_GOOGLE_SECRET="GOCSPX-your-google-client-secret"

# Database URL for Prisma
DATABASE_URL="postgresql://user:password@localhost:5432/mydb?schema=public"
```

Google Provider

Step 4: Assign Google Provider into auth.ts

- In Auth.js v5, environment variables starting with:

AUTH_GOOGLE_ID

AUTH_GOOGLE_SECRET

These are

automatically inferred

by the Google provider!

TS auth.ts > ...

```ts
import NextAuth from "next-auth";
import Google from "next-auth/providers/google";

export const { handlers, signIn, signOut, auth } = NextAuth({
  providers: [Google],
  callbacks: {
    authorized: async ({ auth }) => {
      // Logged in users are authenticated, otherwise redirect to login page
      return !!auth;
    }
  }
});

```

Google Provider

Step 5: Code the server action

Use auth auth, signIn and signOut

app > actions > TS GoogleAction.ts > ...

```ts
"use server";

import { auth, signIn, signOut } from "@/auth";

export async function handleGoogleLogin() {
  console.log(`google action`);
  await signIn("google", { redirectTo: "/" });
}

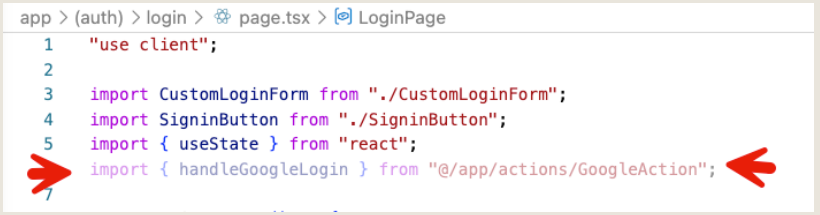
export async function checkGoogleSession() {
  const session = await auth();
  return session;
}

export async function handleGoogleSignout() {
  await signOut({ redirectTo: "/login" });
}

```

Google Provider

Step 6: apply to login page





Pass the action to `formAction` (if attach to a button, or action if attach to form)

Note: button must be of type submit

`formAction` allows individual buttons inside that form to override the default action when clicked.

Create a catch all route handler

- Create the API route handler that processes OAuth callbacks, tokens, and sessions:

```
// app/api/auth/[...nextauth]/route.ts
import { handlers } from '@/auth';

export const { GET, POST } = handlers;
```

Custom hook

- For the `NavBar` to check if there is a current login user
- Provide login/logout appropriately!

```
<div className="navbar-end">
  {isAuthenticated ? (
    <div className="dropdown dropdown-end">
      <div
        tabIndex={0}
        role="button"
        className="btn btn-ghost btn-circle avatar placeholder"
      >
        <div className="bg-primary text-primary-content w-10 rounded-full flex items-
          <span className="text-lg">
            {user?.name?.charAt(0).toUpperCase()}
          </span>
        </div>
      </div>
      <ul
        tabIndex={0}
        className="menu menu-sm dropdown-content bg-base-100 rounded-box z-1 mt-3 w-5
      >
        <li className="menu-title">{user?.name}</li>
        <li>
          <button onClick={logout}>Logout</button>
        </li>
      </ul>
    </div>
  ) : (
    <div className="flex gap-2">
      <Link href="/login" className="btn btn-ghost btn-sm">
        Login
      </Link>
      <Link href="/register" className="btn btn-primary btn-sm">
        Sign Up
      </Link>
    </div>
  )}
```

useAuth hook

- Custom hook

app > hooks > useAuth.tsx > AuthState > user

```
1   "use client";
2
3   import {
4     createContext,
5     useContext,
6     useState,
7     useEffect,
8     useCallback,
9     ReactNode
10  } from "react";
11
12  import {
13    checkGoogleSession,
14    handleGoogleSignout
15  } from "../actions/GoogleAction";
16
```

useAuth hook

- States we need
- Create the context

```
18
19  interface User {
20    id: string | null | undefined;
21    email: string | null | undefined;
22    name: string | null | undefined;
23  }
24
25  interface AuthState {
26    user: User;
27    isAuthenticated: boolean;
28    isLoading: boolean;
29  }
30
31  interface AuthContextType extends AuthState {
32    checkGoogle: () => void;
33    logout: () => Promise<void>;
34  }
```

```
35
36  const AuthContext = createContext<AuthContextType | undefined>(undefined);
37
```

useAuth hook

- Provider function
- `useState` hook to initialise the state (user, isAuthenticated, isLoading)
- Async function to check Google session

```
35
36  export function AuthProvider({ children }: { children: ReactNode }) {
37    const user = {
38      id: "",
39      email: "",
40      name: ""
41    };
42
43    const [authState, setAuthState] = useState<AuthState>({
44      user: user,
45      isAuthenticated: false,
46      isLoading: true
47    });
48
49    const checkGoogle = async () => {
50      const session = await checkGoogleSession();
51      if (session) {
52        const currentUser: User = {
53          id: session.user?.id,
54          name: session.user?.name,
55          email: session.user?.email
56        };
57        setAuthState({
58          user: currentUser,
59          isAuthenticated: true,
60          isLoading: false
61        });
62      }
63    };
64
```

useAuth hook

Execute the async function to check Google session once on mount

Code logout function using `useCallback` hook, again it is async

Return React Context provider with our custom value

```
64
65    // Check auth status on mount
66    useEffect(() => {
67      checkGoogle();
68    }, []);
69
70    const logout = useCallback(async () => {
71      setAuthState({
72        user: user,
73        isAuthenticated: false,
74        isLoading: false
75      });
76
77      await handleGoogleSignout();
78    }, []);
79
80    const value: AuthContextType = {
81      ...authState,
82      checkGoogle,
83      logout
84    };
85
86    return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
87  }
88
```

useAuth hook

Export the hook!

```
88
89  /**
90   * useAuth hook - uses shared AuthContext
91   * All components using this hook share the same auth state
92   */
93  export function useAuth() {
94    const context = useContext(AuthContext);
95    if (context === undefined) {
96      throw new Error("useAuth must be used within an AuthProvider");
97    }
98    return context;
99  }
100
```

We also need a Provider wrapper

app > components > Providers.tsx > ...

```tsx
"use client";

import { AuthProvider } from "../hooks/useAuth";

export function Providers({ children }: { children: React.ReactNode }) {
  return <AuthProvider>{children}</AuthProvider>;
}

```

We also need a Provider wrapper

- Then wrap it in `layout.tsx`
- We can now use this anywhere in the project

```
7   import { Providers } from "./components/Providers";
8
```

```
34        <body className="min-h-full flex flex-col">
35          <Providers>
36            <PeopleProvider>
37              <Toaster position="bottom-right" />
38              <NavBar />
39              {children}
40            </PeopleProvider>
41          </Providers>
42        </body>
```

Apply `useAuth` hook inside the navbar component

app > components > NavBar.tsx > NavBar

```tsx
"use client";

import { Links } from "./navbarData";
import Link from "next/link";
import { usePathname } from "next/navigation"; // for checking active links
import { useAuth } from "../hooks/useAuth";

const NavBar = () => {
  const pathname = usePathname();
  const { isAuthenticated, user, logout, isLoading } = useAuth();
  console.log(`isAuthenticated ... ${isAuthenticated}`);
```

Apply `useAuth` hook inside the navbar component

```
const NavBar = () => {
  const pathname = usePathname();
  const { isAuthenticated, user, logout, isLoading } = useAuth();
  console.log(`isAuthenticated ... ${isAuthenticated}`);
```

```
{isAuthenticated ? (
  <div className="dropdown dropdown-end">
    <div
      tabIndex={0}
      role="button"
      className="btn btn-ghost btn-circle avatar placeholder"
    >
      <div className="bg-primary text-primary-content w-10 ro
        <span className="text-lg">
          {user?.name?.charAt(0).toUpperCase()}
        </span>
      </div>
    </div>
    <ul
      tabIndex={0}
      className="menu menu-sm dropdown-content bg-base-100 ro
    >
      <li className="menu-title">{user?.name}</li>
      <li>
        <button onClick={logout}>Logout</button>
      </li>
    </ul>
```

Protecting Routes via Proxy

- Protect routes automatically before they render by using Next.js proxy middleware:

```
import { auth } from "@/auth";

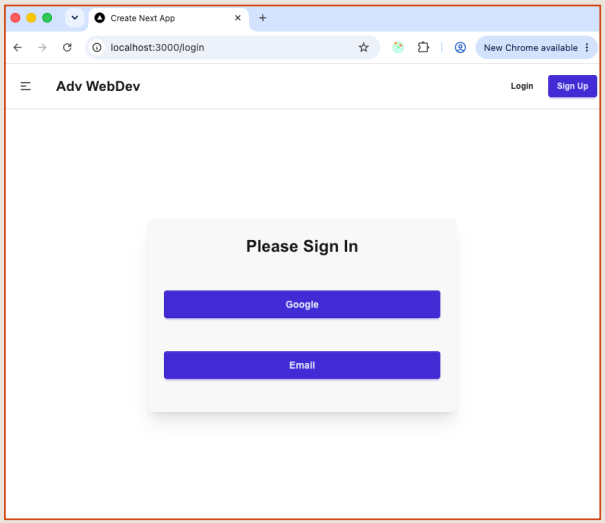
export const proxy = auth((req) => {
  if (!req.auth && req.nextUrl.pathname !== "/login") {
    const newUrl = new URL("/login", req.nextUrl.origin);
    return Response.redirect(newUrl);
  }
});

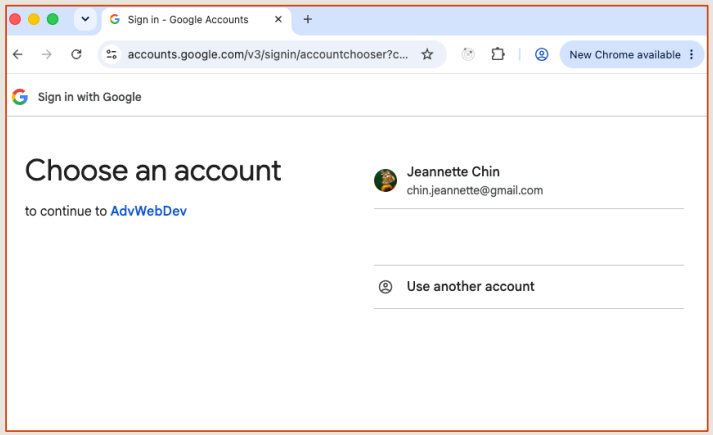
export const config = {
  matcher: ["/((?!api|_next/static|_next/image|favicon.ico).*)"]
};

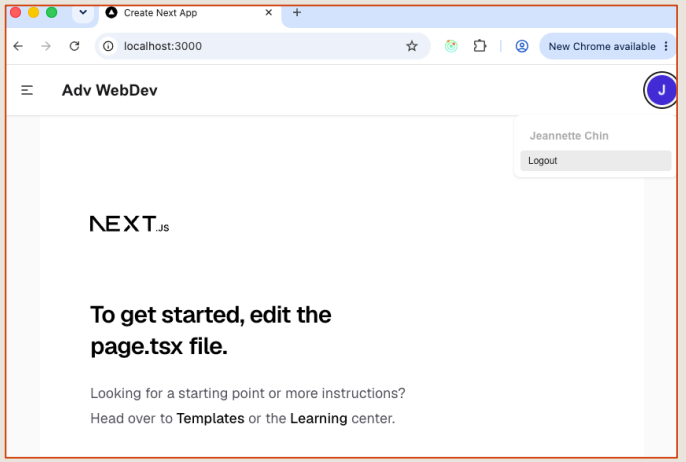
```

Google Provider - Test it!

- **Step 7: Test the Integration**
- Run your development server and test it!







Proxy further configurations

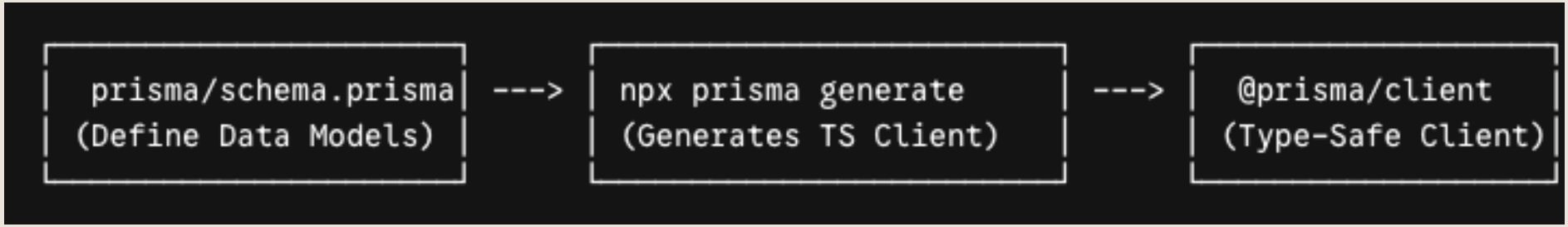
```
12  export const proxy = auth((req) => {
13    const { pathname } = req.nextUrl;
14    const isAuthRoute = pathname === "/login" || pathname === "/register";
15
16    // If user is not authenticated and not visiting an auth page, redirect to login
17    if (!req.auth && !isAuthRoute) {
18      const newUrl = new URL("/login", req.nextUrl.origin);
19      return Response.redirect(newUrl);
20    }
21
22    // (Optional) If the user IS authenticated, prevent them from visiting login/register
23    if (req.auth && isAuthRoute) {
24      const homeUrl = new URL("/", req.nextUrl.origin);
25      return Response.redirect(homeUrl);
26    }
27  });
28
```

Prisma ORM

ORM

- An **ORM (Object-Relational Mapping)** maps database tables directly to TypeScript classes and types, removing the need to write raw SQL strings.
- **Key Benefits**
- **Type-Safe Database Access:** Prisma generates a custom TypeScript client based on your database schema.
- **Declarative Migrations:** Define your database models once in a readable .prisma file, and Prisma handles generating SQL migrations automatically.

Prisma Architecture Overview

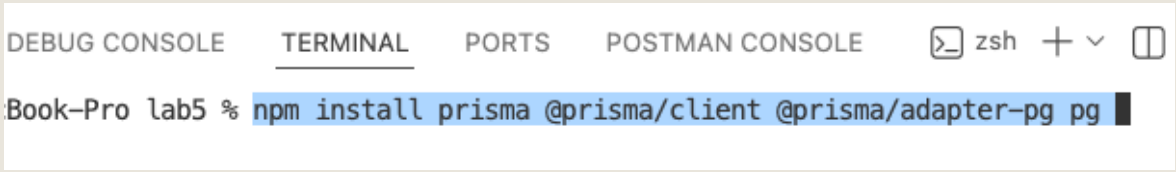


Teaching code – Part2

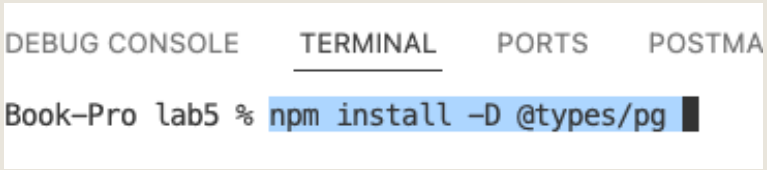
https://stugit.cmp.uea.ac.uk/jc/Adv-WebDev-26-27-W4-part2.git

Prisma step by step

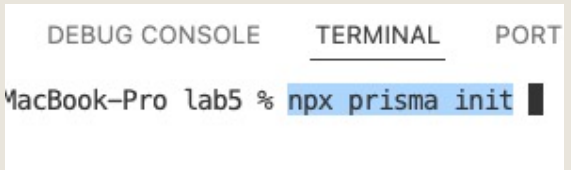
- Step 1: Initialize Prisma & Install Dependencies



- Install pg type



- initialise Prisma in the project:



Prisma step by step

- Step 2: Configure prisma/schema.prisma

```
Generate
generator client {
  provider = "prisma-client"
  output   = "../lib/generated/prisma"
}

datasource db {
  provider = "postgresql"
}

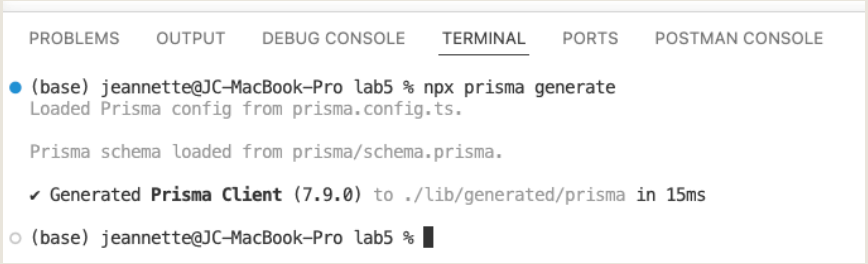

model User {
  id        Int      @id @default(autoincrement())
  email     String   @unique
  password  String
  name      String
  createdAt DateTime @default(now()) @map("created_at")
  updatedAt DateTime @updatedAt @map("updated_at")


  @@map("users")
}

```

Prisma step by step

- Step 3: Generate Prisma client



- Create Prisma client to connect to the database

app > lib > TS prisma.ts > ...

```ts
import { PrismaClient } from "@/lib/generated/prisma/client";
import { PrismaPg } from "@prisma/adapter-pg";
import { Pool } from "pg";

const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined;
};

function createPrismaClient() {
  const pool = new Pool({ connectionString: process.env.DATABASE_URL });
  const adapter = new PrismaPg(pool);
  return new PrismaClient({ adapter });
}

export const prisma = globalForPrisma.prisma ?? createPrismaClient();

if (process.env.NODE_ENV !== "production") {
  globalForPrisma.prisma = prisma;
}

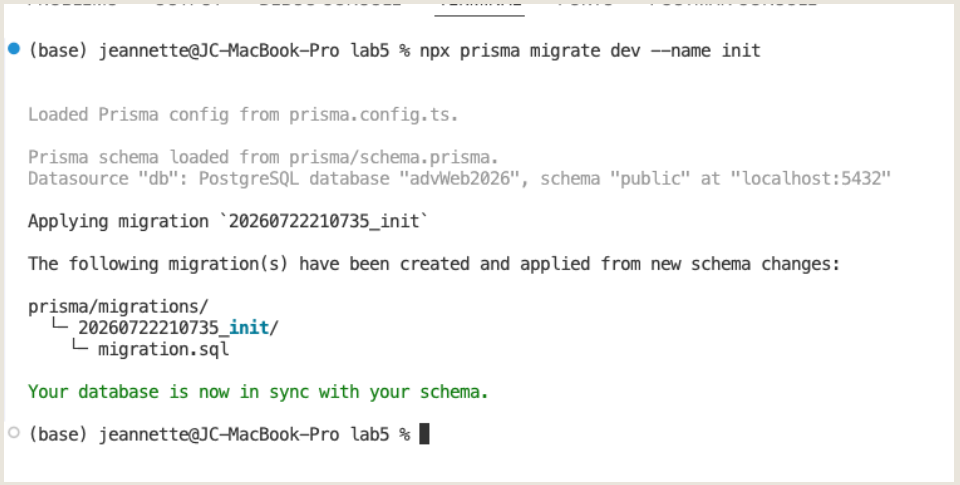
```

Prisma step by step

- Step 4: Generate migration
- If you see error, change the URL in your .env file
- Check your migration file







Reset migration

- **Simply type** `npx prisma migrate reset` and `Y (Yes)`

```
(base) jeannette@JC-MacBook-Pro lab5 % npx prisma migrate reset
Loaded Prisma config from prisma.config.ts.

Prisma schema loaded from prisma/schema.prisma.
Datasource "db": PostgreSQL database "advWeb2026", schema "public" at "localhost:5432"

✔ Are you sure you want to reset your database? All data will be lost. … yes

Database reset successful
```

Now we need to write some functions to test it!

Helper server actions

- Add user
- Edit user
- Remove user
- Get all users

app > actions > TS UserAction.ts > addUser > newUser > dat

```
1   "use server";
2
3   import { prisma } from "../lib/prisma";
4   import { revalidatePath } from "next/cache";
5
```

```
6   // --- 1. ADD USER ---
7   export async function addUser(data: {
8     name: string;
9     email: string;
10    pass: string;
11  }) {
12    try {
13      const newUser = await prisma.user.create({
14        data: {
15          name: data.name,
16          email: data.email,
17          password: data.pass,
18        },
19      });
20
21      // Refresh the page cache to display the new user immediately
22      revalidatePath("/users");
23      return { success: true, user: newUser };
24    } catch (error) {
25      console.error("Failed to add user:", error);
26      return {
27        success: false,
28        error: "Failed to create user. Email may already exist.",
29      };
30    }
31  }
```

Helper server

- Add user
- Edit user
- Remove user
- Get all users

app > actions > TS UserAction.ts > addUser > newUser > dat

```
32
33  // --- 2. EDIT USER ---
34  export async function editUser(
35    id: number,
36    data: { name?: string; email?: string; pass?: string },
37  ) {
38    try {
39      const updatedUser = await prisma.user.update({
40        where: { id },
41        data: {
42          ...(data.name && { name: data.name }),
43          ...(data.email && { email: data.email }),
44          ...(data.pass && { password: data.pass }),
45        },
46      });
47
48      revalidatePath("/users");
49      return { success: true, user: updatedUser };
50    } catch (error) {
51      console.error("Failed to update user:", error);
52      return { success: false, error: "Failed to update user." };
53    }
54  }
55
```

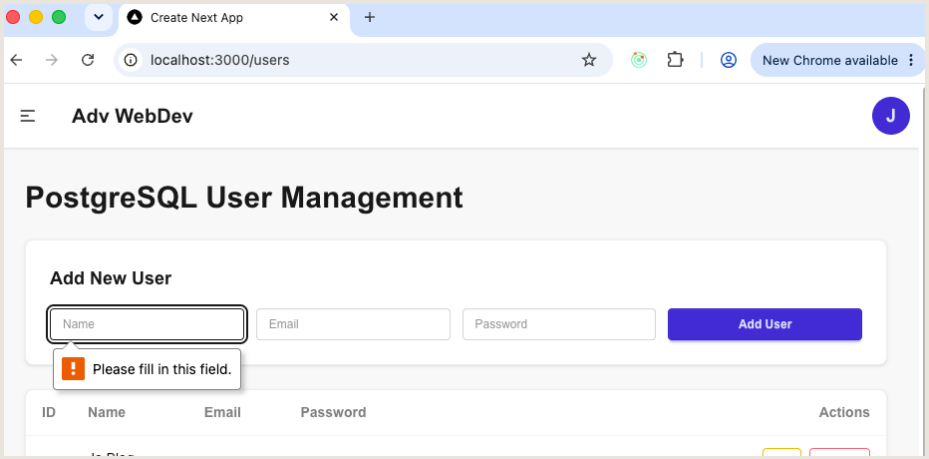
Helper server

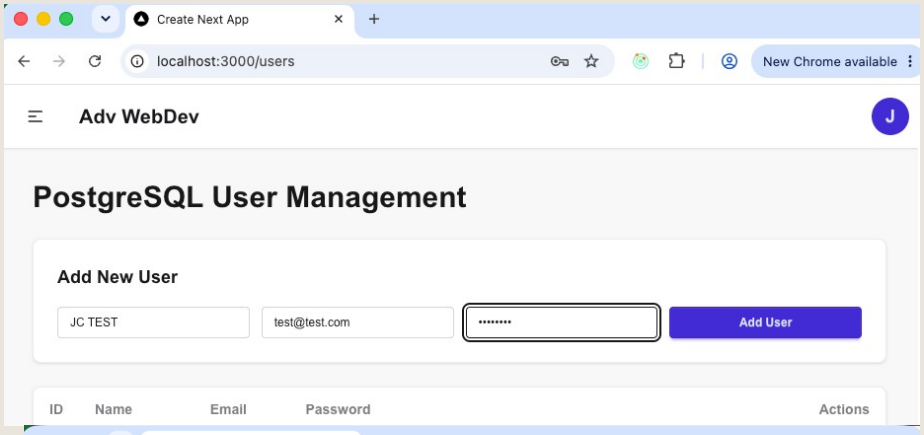
- Add user
- Edit user
- Remove user
- Get all users

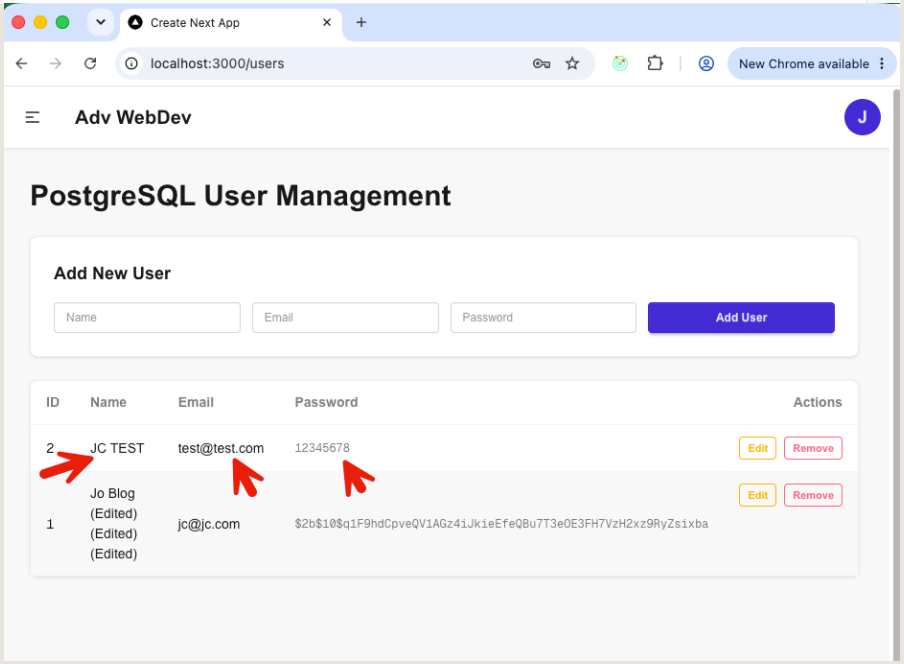
```
55
56  // --- 3. REMOVE USER ---
57  export async function removeUser(id: number) {
58    try {
59      const deletedUser = await prisma.user.delete({
60        where: { id },
61      });
62
63      revalidatePath("/users");
64      return { success: true, user: deletedUser };
65    } catch (error) {
66      console.error("Failed to remove user:", error);
67      return { success: false, error: "Failed to remove user." };
68    }
69  }
70
71  // --- 4. GET ALL USERS (Helper to render the list) ---
72  export async function getUsers() {
73    return await prisma.user.findMany({
74      orderBy: { id: "desc" },
75    });
76  }
77
```

users/page.tsx

- Code add user form
- Code a table to render the user!







Form validation

- To handle form validation and pending state feedback cleanly, React provides two built-in hooks that work seamlessly with Next.js Server Actions:
- **`useActionState`** — Captures returned validation errors, status messages, and manages the form submission state.
- **`useFormStatus`** — Placed inside the submit button component to access the pending state and render a loading spinner automatically.
- E.g. use **Zod** for schema validation and **daisyUI** for feedback UI.
- `npm install zod`

Example workflow:

- **User Submits Form:** Clicking the submit button triggers `useFormStatus()`, instantly disabling the button and showing the daisyUI .loading-spinner.
- **Validation:** `addUserAction` runs on the server. If Zod validation fails, it immediately returns targeted error arrays (state.errors).
- **UI Feedback:** `useActionState` receives the returned state and updates the component, rendering field-level red border alerts and text messages without losing user context.
- **Success:** Upon valid entry, Prisma writes to PostgreSQL, `revalidatePath('/users')` refreshes the page list, and a success banner displays.

PostgreSQL User Management

Add New User using Zod Validation

Validation failed. Please check your inputs.

Name

Jane Doe

Name must be at least 2 characters

Email

jane@example.com

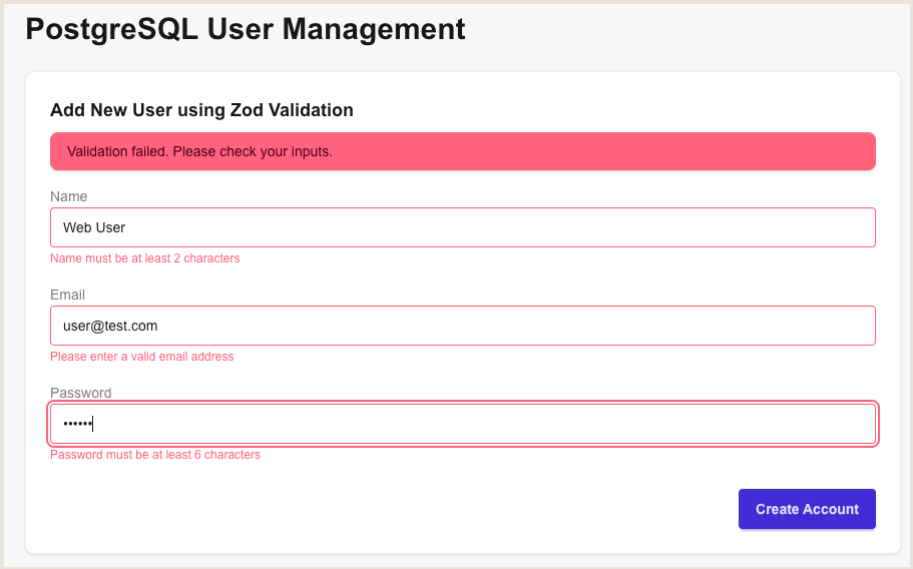
Please enter a valid email address

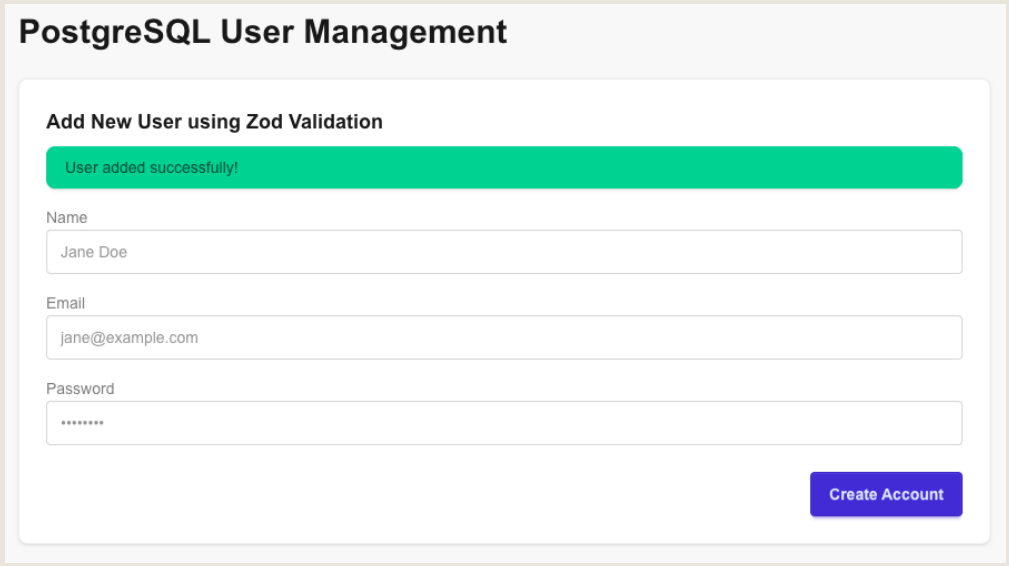
Password

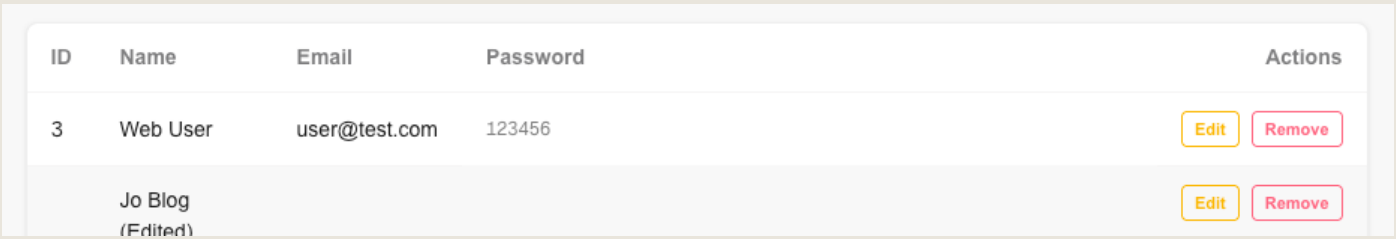
••••••••

Password must be at least 6 characters

Create Account


PostgreSQL User Management
Add New User using Zod Validation
Validation failed. Please check your inputs.
Name
Web User
Name must be at least 2 characters
Email
user@test.com
Please enter a valid email address
Password
Password must be at least 6 characters
Create Account


PostgreSQL User Management
Add New User using Zod Validation
User added successfully!
Name
Jane Doe
Email
jane@example.com
Password
Create Account


ID
Name
Email
Password
Actions
3
Web User
user@test.com
123456
Edit
Remove
Jo Blog
(Edited)
Edit
Remove

API end points

- In Next.js App Router, the `app/api` directory is a convention used to organise **pure API endpoints** (often called **Route Handlers**).
- Instead of returning visual UI (like React HTML components), files in this directory return raw data (usually JSON, text, streams, or files) using standard HTTP methods.

- **The Rule:** You **cannot** put `page.tsx` and `route.ts` in the exact same folder! They represent the same route URL, so having both causes a routing conflict error in Next.js.

How the Folder Structure Maps to Endpoints

- Folder paths directly map to HTTP endpoint URLs:

```
app/
├── api/
│   ├── health/
│   │   └── route.ts          ──> GET/POST  http://localhost:3000/api/health
│   ├── users/
│   │   ├── route.ts          ──> GET/POST  http://localhost:3000/api/users
│   │   └── [id]/
│   │       └── route.ts      ──> GET/DELETE http://localhost:3000/api/users/123
│   └── auth/
│       └── [...nextauth]/
│           └── route.ts      ──> Handles OAuth callbacks (Auth.js)
```

How a `route.ts` File Works

- Inside a `route.ts` file, you **export functions named after the HTTP methods** (GET, POST, PUT, DELETE, PATCH) you want to handle.

```ts
import { NextRequest, NextResponse } from 'next/server';

// Handles GET /api/users
export async function GET(request: NextRequest) {
  const users = [
    { id: 1, name: 'Alice' },
    { id: 2, name: 'Bob' },
  ];

  return NextResponse.json(users, { status: 200 });
}

// Handles POST /api/users
export async function POST(request: NextRequest) {
  const body = await request.json();

  // Perform database insert here (e.g., via Prisma)

  return NextResponse.json(
    { message: 'User created successfully', data: body },
    { status: 201 }
  );
}
```

Why provide Route handlers?

While `Next.js` Server Actions can handle many form submissions directly, `app/api` Route Handlers are still essential for:

- **External Webhooks:** Receiving events from third parties like Stripe (/api/webhooks/stripe), GitHub, or Lemon Squeezy.
- **Public / Mobile APIs:** Exposing REST endpoints to external services, mobile apps (iOS/Android), or public clients.
- **Authentication Handlers:** Auth.js / NextAuth uses `app/api/auth/[...nextauth]/route.ts` to manage Google OAuth callbacks and token exchanges.
- **Streaming Data / LLMs:** Streaming AI text responses (e.g., OpenAI or Vercel AI SDK streams) or Server-Sent Events (SSE).

Exposing API end points

- Can be a major security risk if left unprotected!
- When you place a `route.ts` inside `app/api`, anyone on the internet can send HTTP requests (GET, POST, DELETE, etc.) directly to that URL using tools like Postman, curl, or automated malicious scripts, completely bypassing your visual frontend interface.

- However, exposed APIs are necessary for web applications to communicate!

- The goal isn't to hide the endpoint, but to **defend and harden it** so unauthorised users cannot abuse it.

The Core Security Risks

- **Unauthorised Access & Data Leaks:** An attacker hits `/api/users/123` and reads sensitive user data because the endpoint doesn't check who is asking.
- **Data Tampering (Bypassing UI Rules):** Your React form might disable the "Submit" button if a field is invalid, but an attacker can skip your form and POST invalid or malicious JSON straight to `/api/data`.
- **Denial of Service (DoS) & Resource Exhaustion:** Attackers loop 100,000 requests to `/api/search`, crashing your database or inflating your cloud server bills.
- **SQL Injection / Data Corruption:** Unsanitised incoming request data executed directly against your database.

How to Secure Next.js `app/api` Routes (The Defence Playbook)

1. Authentication & Authorisation (Who are you? What can you do?)

- Never assume a request came from your app's frontend. Always check session credentials inside the handler or via middleware

2. Strict Input Validation (Schema Guarding)

- Use validation libraries like **Zod** to validate and sanitise every incoming request body before hitting your database.

3. Rate Limiting (Preventing Abuse)

- Protect your routes from brute-force attacks and bot spam using rate limiters (such as @upstash/ratelimit via Redis or proxy headers)

How to Secure Next.js `app/api` Routes (The Defence Playbook)

4. Alternative: Use Server Actions for Internal App Forms

- If your API endpoints exist *only* to feed forms inside your own Next.js application, consider using **Server Actions** ('use server') instead of standard `app/api` routes.
- Server Actions generate cryptographically signed action IDs, reducing the need to manually build public REST endpoints for simple internal mutations.

Exposing an API route is completely standard practice in web development, provided you treat every incoming request as **untrusted by default**. Validate inputs, enforce authentication/authorisation, and rate-limit your routes!

Practical Exercise and homework

- Lab 4 – Next Auth with Google Provider
- Lab 4.1 – Prisma ORM

Next Session

- Custom Credentials with Auth.js and putting everything together